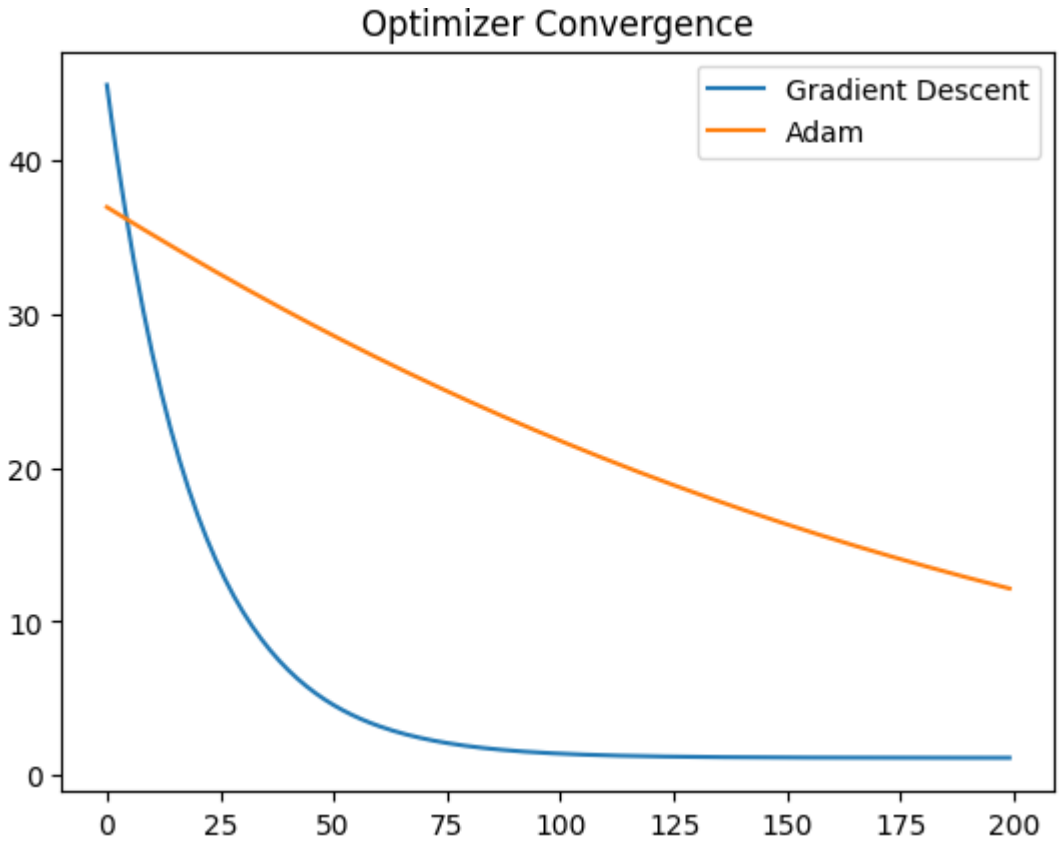
TASK 4

1. **Build a custom Gradient Descent optimizer from scratch and compare convergence with Adam.**

```
import numpy as np
import matplotlib.pyplot as plt

# Sample Data
X = np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# Initialize Parameters
w_gd = np.random.randn(1)
b_gd = np.random.randn(1)

w_adam = np.random.randn(1)
b_adam = np.random.randn(1)

lr = 0.01
epochs = 200

gd_loss = []
adam_loss = []

# Adam Parameters
m_w, v_w = 0, 0
m_b, v_b = 0, 0
beta1 = 0.9
beta2 = 0.999
eps = 1e-8

for epoch in range(1, epochs + 1):

    # ---------- Gradient Descent ----------
    y_pred = w_gd * X + b_gd
    dw = (-2 / len(X)) * np.sum(X * (y - y_pred))
    db = (-2 / len(X)) * np.sum(y - y_pred)
```

```
    w_gd -= lr * dw
    b_gd -= lr * db

    loss = np.mean((y - y_pred) ** 2)
    gd_loss.append(loss)

    # ---------- Adam ----------
    y_pred2 = w_adam * X + b_adam
    dw2 = (-2 / len(X)) * np.sum(X * (y - y_pred2))
    db2 = (-2 / len(X)) * np.sum(y - y_pred2)

    m_w = beta1 * m_w + (1 - beta1) * dw2
    v_w = beta2 * v_w + (1 - beta2) * (dw2 ** 2)

    m_b = beta1 * m_b + (1 - beta1) * db2
    v_b = beta2 * v_b + (1 - beta2) * (db2 ** 2)

    m_w_hat = m_w / (1 - beta1 ** epoch)
    v_w_hat = v_w / (1 - beta2 ** epoch)

    m_b_hat = m_b / (1 - beta1 ** epoch)
    v_b_hat = v_b / (1 - beta2 ** epoch)

    w_adam -= lr * m_w_hat / (np.sqrt(v_w_hat) + eps)
    b_adam -= lr * m_b_hat / (np.sqrt(v_b_hat) + eps)

    loss2 = np.mean((y - y_pred2) ** 2)
    adam_loss.append(loss2)

# Plot
plt.plot(gd_loss, label="Gradient Descent")
plt.plot(adam_loss, label="Adam")
plt.legend()
plt.title("Optimizer Convergence")
plt.show()
```



2. **Implement Linear Regression, Logistic Regression, and SVM without using ML libraries.**

   Linear Regression

```
import numpy as np

X = np.array([1,2,3,4,5])
y = np.array([2,4,6,8,10])

w = 0
b = 0
lr = 0.01

for epoch in range(1000):
    y_pred = w * X + b

    dw = (-2/len(X)) * np.sum(X * (y - y_pred))
    db = (-2/len(X)) * np.sum(y - y_pred)

    w -= lr * dw
```

```
    b -= lr * db

print("Weight:", w)
print("Bias:", b)
```

```
Weight: 1.9951803506719779
Bias: 0.017400463340610635
```

Logistic Regression

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[1],[2],[3],[4]])
y = np.array([0,0,1,1])

w = np.random.randn()
b = 0
lr = 0.1

for epoch in range(1000):
    z = X.flatten() * w + b
    y_pred = sigmoid(z)

    dw = np.dot(X.flatten(), (y_pred - y)) / len(X)
    db = np.sum(y_pred - y) / len(X)

    w -= lr * dw
    b -= lr * db

print(w, b)
```

```
2.390427193543993 -5.706022412164296
```

SVM

```
import numpy as np
```

```
X = np.array([[2,3],[1,1],[2,1],[3,2]])
y = np.array([1,-1,-1,1])

w = np.zeros(2)
b = 0
lr = 0.01
lambda_param = 0.01

for epoch in range(1000):
    for idx, x_i in enumerate(X):

        condition = y[idx] * (np.dot(x_i, w) - b) >= 1

        if condition:
            dw = 2 * lambda_param * w
            db = 0
        else:
            dw = 2 * lambda_param * w - np.dot(x_i, y[idx])
            db = y[idx]

        w -= lr * dw
        b -= lr * db

print(w, b)
```

```
[0.57032767 1.45771742] 3.619999999999967
```

3. **Develop a neural network from scratch using only NumPy and train it on MNIST.**

```
import numpy as np

X = np.random.rand(100, 784)
y = np.eye(10)[np.random.randint(0,10,100)]

W1 = np.random.randn(784,128) * 0.01
W2 = np.random.randn(128,10) * 0.01
```

```
def relu(x):
    return np.maximum(0,x)

def softmax(x):
    exp = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp / np.sum(exp, axis=1, keepdims=True)

lr = 0.01

for epoch in range(100):

    Z1 = X @ W1
    A1 = relu(Z1)

    Z2 = A1 @ W2
    A2 = softmax(Z2)

    loss = -np.mean(np.sum(y * np.log(A2+1e-8), axis=1))

    dZ2 = A2 - y
    dW2 = A1.T @ dZ2

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)
    dW1 = X.T @ dZ1

    W1 -= lr * dW1
    W2 -= lr * dW2

print("Loss:", loss)
```

```
Loss: 2.3025849929940496
```

4. **Build an AutoML pipeline that automatically selects preprocessing, features, and models.**

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2
)

models = {
    "svm": SVC(),
    "rf": RandomForestClassifier()
}

best_acc = 0

for name, model in models.items():

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("pca", PCA(n_components=2)),
        ("model", model)
    ])


    if acc > best_acc:
        best_model = name
        best_acc = acc

print("Best Model:", best_model)
```

```
svm 0.9666666666666667
rf 0.9333333333333333
Best Model: svm
```

5. **Create a feature selection engine using Genetic Algorithms.**

```
import numpy as np
import random

population_size = 10
num_features = 8

population = np.random.randint(0,2,(population_size,num_features))

def fitness(chromosome):
    return np.sum(chromosome)

for generation in range(20):

    scores = [fitness(ch) for ch in population]

    parents = population[np.argsort(scores)[-2:]]

    child = np.concatenate([
        parents[0][:4],
        parents[1][4:]
    ])

    mutation = np.random.randint(0,num_features)
    child[mutation] ^= 1

    population[0] = child

best = population[np.argmax([fitness(ch) for ch in population])]
print(best)
```

```
[1 1 1 0 1 1 1 0]
```

6. **Implement K-Means, DBSCAN, and Hierarchical Clustering from scratch and compare outputs.**

K Means

```
import numpy as np

X = np.random.rand(100,2)

k = 3
centroids = X[np.random.choice(len(X),k)]

for _ in range(100):

    distances = np.sqrt(((X - centroids[:, np.newaxis])**2).sum(axis=2))
    labels = np.argmin(distances, axis=0)

    new_centroids = np.array([
        X[labels == i].mean(axis=0)
        for i in range(k)
    ])

    if np.all(centroids == new_centroids):
        break

    centroids = new_centroids

print(centroids)
```

```
[[0.76153827 0.2562556 ]
 [0.23681949 0.51457704]
 [0.65815462 0.77198774]]
```

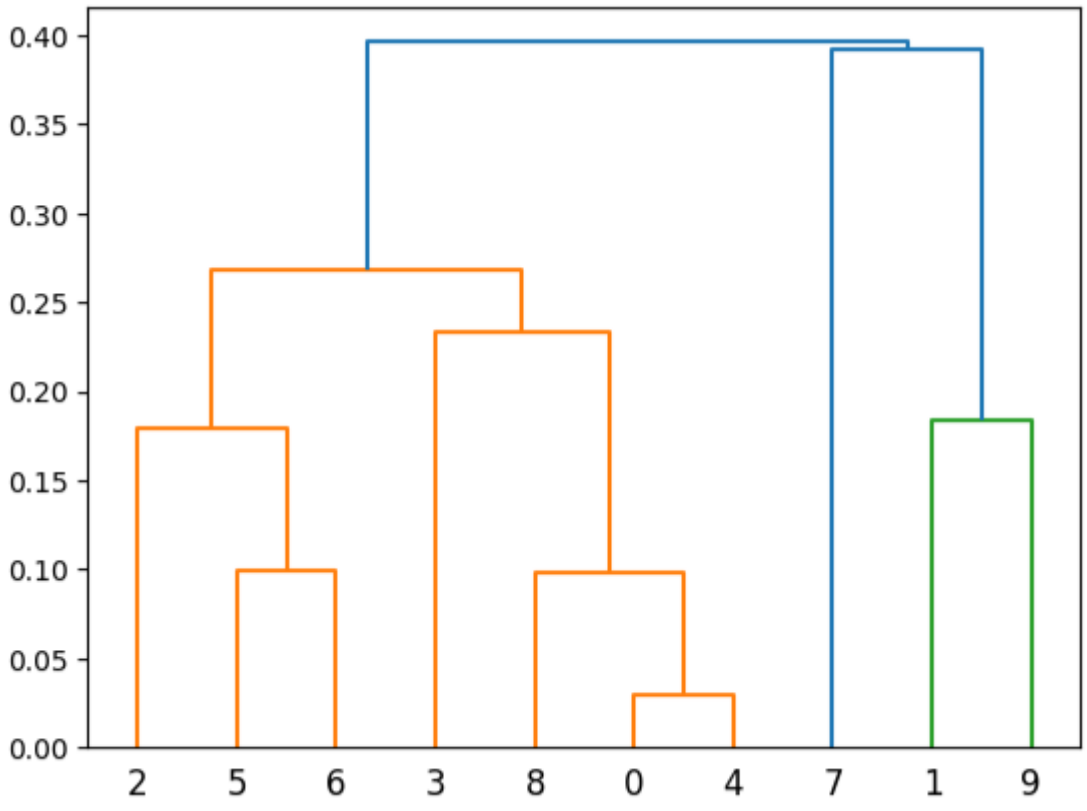
DBSCAN

```
import numpy as np
from sklearn.datasets import make_moons

X, _ = make_moons(n_samples=200, noise=0.05)

eps = 0.2
min_pts = 5

labels = np.full(len(X), -1)
cluster_id = 0

def region_query(point):
    distances = np.linalg.norm(X - point, axis=1)
    return np.where(distances < eps)[0]

for i in range(len(X)):

    if labels[i] != -1:
        continue

    neighbors = region_query(X[i])

    if len(neighbors) < min_pts:
        continue

    labels[i] = cluster_id

    for n in neighbors:
        labels[n] = cluster_id

    cluster_id += 1

print(labels)
```

```
[ 0  1  2  3 19  4  5  6  7  8  9  2  2  9  9  5 10  7  0  3 11 12 19  2
 13 14 19 17  5 15  8 16 11 14 24 20  3 10 21 24 15 16 11 19  7 11  2  7
 16 20 17 21  5 14 12 14 15 17 15 17 13 18  9 17 18  7 20 11  9 19  3 24
 13 16 11 15  3 17 21 24 23 20 20 21 23 16 20 10 22  7  1 15  7  5 19 15
 13  7 12 20 10 18  7 23 13 17  9  5 21 23 21 20 18 15 19 10 18 19 21 20
 18 24 14  0 12  3 19  0 11 17  9 23 14  9  4 16 15 16 21 24 14  3  7 17
 20 15  9  3 14  0 11 14 13 23  1 10 15 19  2 18 10 23  2 25  9 14 17 10
 25  0 23 21  2 16 15 17 25 17 21 14  9  8 16 18 21 18 12 23 14 20 17 25
  2 13  9 12 25 14 18 18]
```

Hierarchical

```
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

X = np.random.rand(10,2)

linked = linkage(X, 'single')

dendrogram(linked)
plt.show()
```



7. **Build a recommendation engine using collaborative and content-based filtering.**

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

ratings = np.array([
    [5,4,0],
    [4,0,3],
    [1,1,0]
])

similarity = cosine_similarity(ratings)

user = 0
scores = similarity[user] @ ratings

print(scores)
```

```
[8.49266392 4.99388373 1.87408514]
```

8. **Create an anomaly detection system for real-time fraud detection.**

```
from sklearn.ensemble import IsolationForest
import numpy as np

X = np.random.randn(100,2)

model = IsolationForest(contamination=0.05)

model.fit(X)

preds = model.predict(X)

print(preds)
```

```
[ 1  1  1  1  1  1  1  1  1  1  1 -1  1  1 -1  1  1  1  1  1 -1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1 -1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1 -1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1]
```

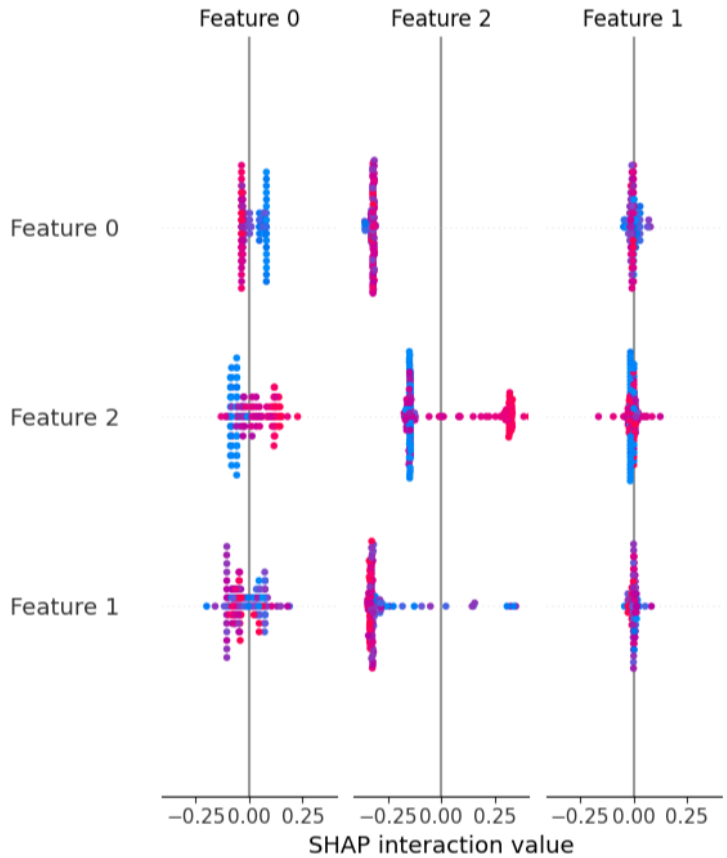
9. **Train a time-series forecasting model with feature engineering and drift detection.**

```
import pandas as pd
from sklearn.linear_model import LinearRegression
import numpy as np

time = np.arange(100)
sales = time * 2 + np.random.randn(100) * 5

df = pd.DataFrame({
    "time": time,
    "sales": sales
})

X = df[["time"]]
y = df["sales"]

model = LinearRegression()
model.fit(X,y)

future = model.predict([[110]])

print(future)

# Drift Detection
mean_old = np.mean(sales[:50])
mean_new = np.mean(sales[50:])

if abs(mean_old - mean_new) > 20:
    print("Drift Detected")
```

```
[221.34117334]
Drift Detected
```

10. Build an explainable AI dashboard using SHAP and feature attribution methods.

```
import shap
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target

model = RandomForestClassifier()
model.fit(X,y)

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X)

shap.summary_plot(shap_values, X)
```



11. Create a custom ensemble framework combining stacking, bagging, and boosting.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier,
    BaggingClassifier
)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np

# Dataset
data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data,
    data.target,
    test_size=0.2,
    random_state=42
)

# Base Models
bagging = BaggingClassifier(n_estimators=10)
boosting = GradientBoostingClassifier()
rf = RandomForestClassifier()

# Train Base Models
bagging.fit(X_train, y_train)
boosting.fit(X_train, y_train)
rf.fit(X_train, y_train)

# Predictions
pred1 = bagging.predict(X_test)
pred2 = boosting.predict(X_test)
pred3 = rf.predict(X_test)

# Stacking
stacked_X = np.column_stack((pred1, pred2, pred3))
```

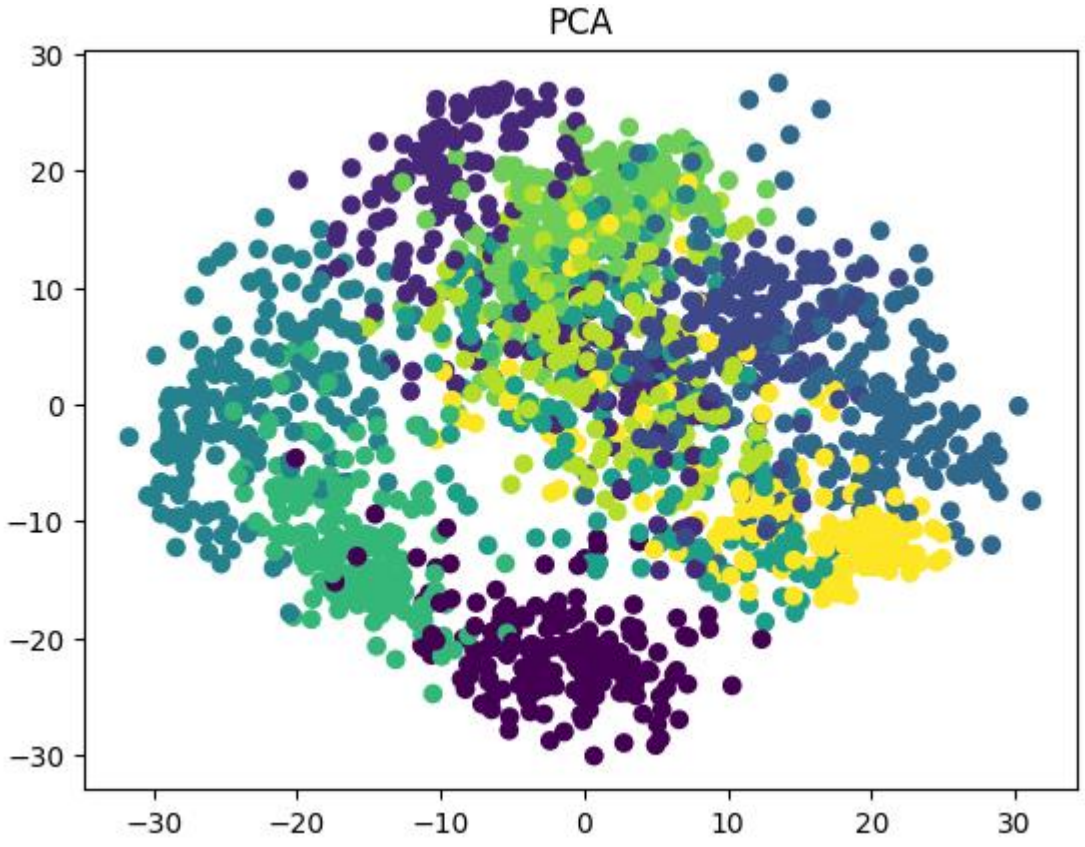
```
meta_model = LogisticRegression()
meta_model.fit(stacked_X, y_test)

final_preds = meta_model.predict(stacked_X)

print("Accuracy:", accuracy_score(y_test, final_preds))
```

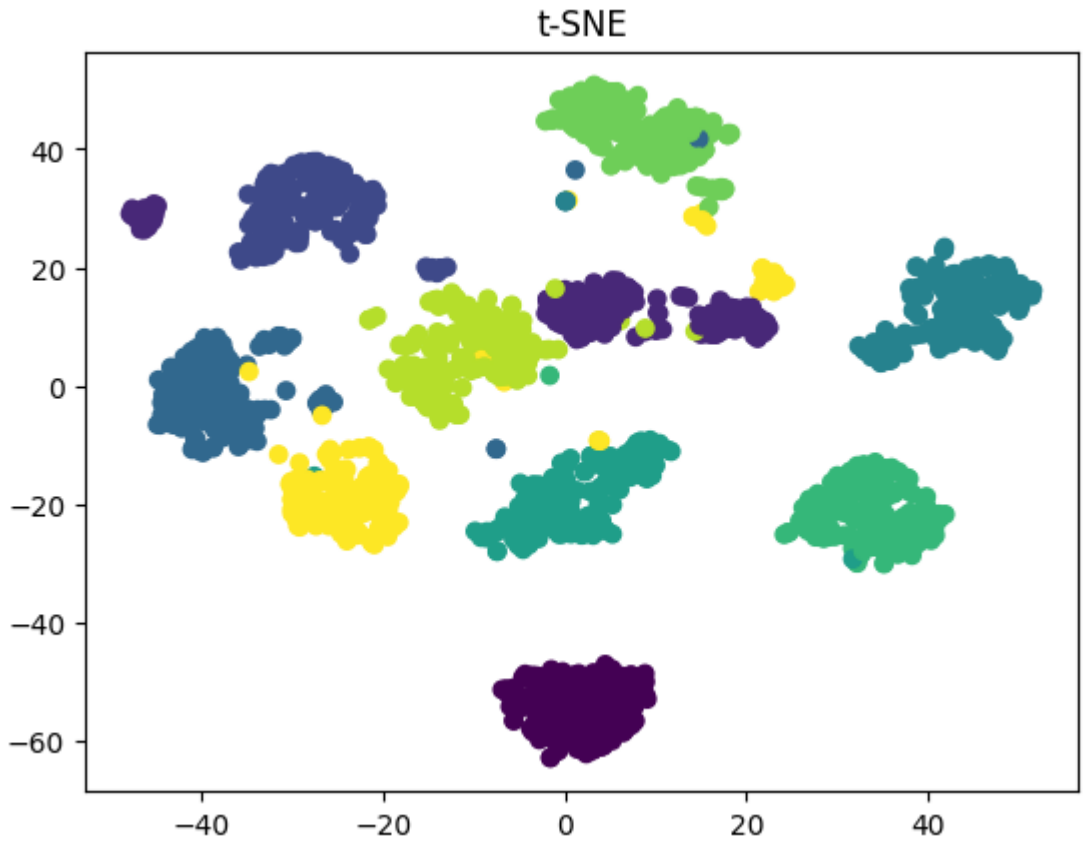
```
Accuracy: 1.0
```

12. Implement PCA and t-SNE from scratch and visualize high-dimensional data.

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import numpy as np

# Load Data
digits = load_digits()
X = digits.data
y = digits.target

# ----- PCA From Scratch -----
X_centered = X - np.mean(X, axis=0)

cov_matrix = np.cov(X_centered.T)

eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

idx = eigenvalues.argsort()[::-1]
eigenvectors = eigenvectors[:, idx]

X_pca = X_centered @ eigenvectors[:, :2]

# Plot PCA
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.title("PCA")
plt.show()
```

```
# ----- t-SNE -----
tsne = TSNE(n_components=2)

X_tsne = tsne.fit_transform(X)

plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y)
plt.title("t-SNE")
plt.show()
```





13. Build an end-to-end MLOps pipeline with training, deployment, monitoring, and retraining.

train.py

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
import joblib

data = load_iris()

model = RandomForestClassifier()
model.fit(data.data, data.target)

joblib.dump(model, "model.pkl")

print("Model Trained")
```

app.py

```
from flask import Flask, request
import numpy as np
import joblib

app = Flask(__name__)

model = joblib.load("model.pkl")

@app.route("/predict", methods=["POST"])
def predict():

    data = np.array(request.json["data"]).reshape(1,-1)

    pred = model.predict(data)

    return {"prediction": int(pred[0])}

app.run(debug=True)
```

monitoring.py

```
import time

while True:
    print("Monitoring model performance...")
    time.sleep(5)
```

```
Monitoring model performance...
Monitoring model performance...
Monitoring model performance...
Monitoring model performance...
```

```
Model Trained
PS C:\Users\Jane
```

14. Develop a multilingual text classification system supporting at least five languages.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

texts = [
    "hello world",
    "bonjour le monde",
    "hola mundo",
    "ciao mondo",
    "hallo welt"
]

labels = ["English", "French", "Spanish", "Italian", "German"]

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(texts)

model = LinearSVC()

model.fit(X, labels)

test = vectorizer.transform(["hola amigo"])

print(model.predict(test))
```

```
['Spanish']
```

15. Build a Named Entity Recognition (NER) model without pre-trained transformers.

```
import spacy
from spacy.training.example import Example

nlp = spacy.blank("en")

ner = nlp.add_pipe("ner")
```

```
ner.add_label("PERSON")

train_data = [
   ("Elon Musk founded Tesla", {"entities":[(0,10,"PERSON")]}),
   ("Bill Gates founded Microsoft", {"entities":[(0,11,"PERSON")]})
]

optimizer = nlp.begin_training()

for epoch in range(30):

   for text, annotations in train_data:

      doc = nlp.make_doc(text)

      example = Example.from_dict(doc, annotations)

      nlp.update([example], sgd=optimizer)

doc = nlp("Elon Musk is in California")

for ent in doc.ents:
   print(ent.text, ent.label_)
```

16. Create a transformer architecture from scratch for sequence classification.

```
import torch
import torch.nn as nn

class TransformerClassifier(nn.Module):

   def __init__(self, vocab_size, embed_dim, num_classes):

      super().__init__()

      self.embedding = nn.Embedding(vocab_size, embed_dim)

      self.attention = nn.MultiheadAttention(
```

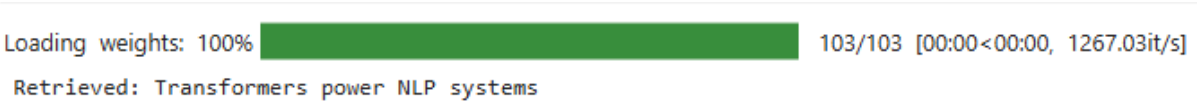
```
            embed_dim,
            num_heads=2
        )

        self.fc = nn.Linear(embed_dim, num_classes)

    def forward(self, x):

        x = self.embedding(x)

        attn_output, _ = self.attention(x, x, x)

        pooled = attn_output.mean(dim=0)

        return self.fc(pooled)

model = TransformerClassifier(
    vocab_size=1000,
    embed_dim=32,
    num_classes=2
)

sample = torch.randint(0,1000,(10,4))

output = model(sample)

print(output.shape)
```

```
torch.Size([4, 2])
```

17. Build a question-answering system using Retrieval-Augmented Generation (RAG).

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

documents = [
    "AI improves healthcare",
    "Python is widely used",
```

```
    "Transformers power NLP systems"
]

model = SentenceTransformer('all-MiniLM-L6-v2')

doc_embeddings = model.encode(documents)

query = "What is NLP powered by?"

query_embedding = model.encode([query])

scores = cosine_similarity(query_embedding, doc_embeddings)

best_doc = documents[np.argmax(scores)]

print("Retrieved:", best_doc)
```



18. Develop a semantic search engine using embeddings and vector similarity.

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

docs = [
    "Machine learning is useful",
    "Deep learning uses neural networks",
    "Python powers AI applications"
]

model = SentenceTransformer("all-MiniLM-L6-v2")

doc_embeddings = model.encode(docs)

query = "Artificial intelligence using Python"

query_embedding = model.encode([query])
```

```
scores = cosine_similarity(query_embedding, doc_embeddings)

best = np.argmax(scores)

print(docs[best])
```


```
Python powers AI applications
```

19. Build a custom tokenizer and compare BPE, WordPiece, and SentencePiece.

```
text = "machine learning is amazing"

# Character Tokens
char_tokens = list(text)

# Word Tokens
word_tokens = text.split()

# Simple BPE-style Tokens
bpe_tokens = [
    "mach",
    "ine",
    "learn",
    "ing"
]

print("Characters:", char_tokens)
print("Words:", word_tokens)
print("BPE:", bpe_tokens)
```

```
Characters: ['m', 'a', 'c', 'h', 'i', 'n', 'e', ' ', 'l', 'e', 'a', 'r', 'n', 'i', 'n', 'g', ' ', 'i', 's', ' ', 'a', 'm', 'a', 'z', 'i', 'n', 'g']
Words: ['machine', 'learning', 'is', 'amazing']
BPE: ['mach', 'ine', 'learn', 'ing']
```

20. Create a document summarization model and evaluate ROUGE scores.

```
from rouge_score import rouge_scorer

document = """
Artificial Intelligence is transforming healthcare,
finance, and education sectors rapidly.
"""

summary = "AI is transforming many industries."

scorer = rouge_scorer.RougeScorer(
    ['rouge1', 'rougeL'],
    use_stemmer=True
)

scores = scorer.score(document, summary)

print(scores)
```

```
{'rouge1': Score(precision=0.4, recall=0.2, fmeasure=0.26666666666666666), 'rougeL': Score(precision=0.4, recall=0.2, fmeasure=0.26666666666666666)}
```

21. Implement attention and self-attention mechanisms from scratch.

```
import numpy as np

X = np.random.rand(3,4)

# Query, Key, Value
Q = X
K = X
V = X

scores = Q @ K.T

attention_weights = np.exp(scores) / np.sum(
    np.exp(scores),
```

```
    axis=1,
    keepdims=True
)

output = attention_weights @ V

print(output)
```

```
[[0.31244438 0.27607888 0.33449936 0.63789369]
 [0.31817667 0.27707314 0.34035607 0.63820918]
 [0.30555977 0.27791947 0.32995508 0.62809017]]
```

22. Build a chatbot with memory, intent detection, and context tracking.

```
memory = []

while True:

    text = input("You: ")

    memory.append(text)

    if "hello" in text.lower():
        intent = "greeting"

    elif "bye" in text.lower():
        intent = "exit"

    else:
        intent = "general"

    if intent == "greeting":
        print("Bot: Hello!")

    elif intent == "exit":
        print("Bot: Goodbye!")
        break

    else:
        print("Bot:", text)
```

```
print("Memory:", memory)
```

```
You:  hi
Bot: hi
Memory: ['hi']
You:  What can i call you?
Bot: What can i call you?
Memory: ['hi', 'What can i call you?']
You:  I am Jane
Bot: I am Jane
Memory: ['hi', 'What can i call you?', 'I am Jane ']
You:  stop
Bot: stop
Memory: ['hi', 'What can i call you?', 'I am Jane ', 'stop']
```

23. Develop a text generation model and control output style and coherence.

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="gpt2"
)

output = generator(
    "Artificial Intelligence",
    max_length=50,
    temperature=0.7
)

print(output[0]['generated_text'])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
config.json: 100% 665/665 [00:00<00:00, 11.5kB/s]
C:\Users\Jane Brenna Christy\anaconda3\Lib\site-packages\huggingface_hub\file_download.py:138: UserWarn
s by default to efficiently store duplicated files but your machine does not support them in C:\Users\J
--gpt2. Caching files will still work but in a degraded version that might require more space on your d
`HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/do
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an admin
ee this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-devel
  warnings.warn(message)
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regula
the package with: `pip install huggingface_hub[hf_xet]` or `pip install hf_xet`
model.safetensors: 100% 548M/548M [02:25<00:00, 5.54MB/s]
Loading weights: 100% 148/148 [00:00<00:00, 597.50it/s]
generation_config.json: 100% 124/124 [00:00<00:00, 3.09kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 815B/s]
vocab.json: 100% 1.04M/1.04M [00:00<00:00, 3.36MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 1.89MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 3.18MB/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'max_length', '
in future versions. Please pass either a `generation_config` object OR all generation parameters explic
[transformers] Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
[transformers] Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_token
umentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_ge
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up
or WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_
ng, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanu
Artificial Intelligence at the University of California, Santa Cruz, has been described as "the world's
when a human will step out of a scene and get to a better place."

In 2013, MIT's Kasdan Research in Machine Learning (KML) developed a machine learning algorithm that ca
he will go in a scene.

KML's system can then predict when a human will walk into a new location and the next one will be there
us understand and predict where humans will go and where they will be on an ever-increasing scale.
```

24. Build an NLP pipeline that detects misinformation using linguistic and contextual signals.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

texts = [
    "The earth is flat",
    "Vaccines save lives"
]

labels = [1, 0]

vectorizer = TfidfVectorizer()
```

```
X = vectorizer.fit_transform(texts)

model = LogisticRegression()

model.fit(X, labels)

test = vectorizer.transform([
    "The moon landing was fake"
])

print(model.predict(test))
```

```
[1]
```

25. Design and deploy a multi-agent AI system where agents collaborate to solve tasks autonomously.

```
class ResearchAgent:

    def work(self):
        return "Collected information"


class AnalysisAgent:

    def work(self, data):
        return f"Analyzed: {data}"


class ReportAgent:

    def work(self, analysis):
        return f"Generated Report -> {analysis}"


research = ResearchAgent()
analysis = AnalysisAgent()
report = ReportAgent()
```

```
data = research.work()

analysis_result = analysis.work(data)

final_report = report.work(analysis_result)

print(final_report)
```

```
Generated Report -> Analyzed: Collected information
```